Android 客户端 + Spring Boot 后端

学生公寓管理系统

双端完整实现 · 作品集

姓名	(your name)
联系方式	(phone / email)
项目类型	课程设计延伸 · 独立开发
客户端	Android（Java 8 + Room + Retrofit）
服务端	Spring Boot 3 + MyBatis + MySQL 8
截图	19 张

项目概览

一个学生公寓管理系统，覆盖学生与管理员两种角色的日常事务：请假审批、报修、进出打卡、卫生评分、缴费、公告、访客登记。

项目的重点不是功能数量，而是两次重构 + 一次前后端联调。

技术栈

客户端	Java 8 · Room 2.4.3 · Retrofit 2.9 · OkHttp 4.12 · RecyclerView · ViewBinding
服务端	Spring Boot 3.2 · MyBatis 3.0 · MySQL 8.0 · H2（演示用）
构建	Gradle 8.9 · AGP 8.7.3 · JDK 17
客户端规模	71 个 Java 文件，约 7200 行，43 个布局
服务端规模	16 个 Java 文件，约 900 行，2 个 MyBatis Mapper XML

三个阶段的演进

阶段	做了什么	解决的痛点
一 · DAO 分层	把散落在各 Activity 里的手写 SQL 与 Cursor 解析收敛成 8 个 DAO	同一段映射代码在 5 个文件里各抄一遍
二 · 迁移 Room	8 个手写 DAO 换成 Room 注解	SQL 写错只能在运行时暴露 → 变成编译期报错
三 · 前后端联调	新增 Spring Boot 后端，客户端用 Retrofit 调用	只能单机使用 → 数据可集中存储

关键技术点

1. 数据访问层收口

改造前每个页面自己开库、自己拼 SQL、自己按列位置取数据：

```
Cursor cursor = db.rawQuery(sql, args);
while (cursor.moveToNext()) {
    Integer id = cursor.getInt(0);        // 列位置硬编码，全项目 89 处
    String account = cursor.getString(1);
}
```

改造后用 `getColumnIndexOrThrow("列名")`，建表字段顺序变化也不会静默串数据。

2. 手写 DAO → Room

	手写 DAO	Room
建表	手写 create table 字符串	由 @Entity 生成，SQL 与字段不可能对不上
写错代价	运行时才炸	编译期报错（把每条 SQL 拿到内存 SQLite 上真跑）
事务	beginTransaction 三件套	一个 @Transaction 注解
联表结果	往实体里塞“有时有值”的字段	@Embedded + 独立 POJO

3. 客户端与服务端打通

客户端用 Retrofit 调用后端 REST 接口。网络开发真正的难点不是“发个请求”，而是**分清三层失败**：

层	表现	典型原因	给用户的提示
传输层	onFailure（ConnectException / 超时）	后端没启动、IP 不对、不同网段	连不上服务器
HTTP 层	isSuccessful() == false	400 参数 / 404 路径 / 500 服务端 bug	按状态码区分
业务层	body().getCode() != 0	权限不足、状态冲突	直接显示后端 message

4. 统一响应与异常语义

HTTP	场景
400	参数校验失败 / 业务规则不满足
404	路径不存在 / 资源不存在

405	请求方法不支持
503	数据库连不上（不是 500）

> 关键取舍：数据库不可用属于**服务端依赖失败**，语义上应返回 503 而非 500； 并且要把原因写进响应体，否则调用方只看到“服务器内部错误”，无从定位。

接口设计与验证

方法	路径	说明
GET	/students?page=&size=	学生列表（分页）
POST	/repairs	新增报修（参数校验 + 只允许学生提交）
GET	/repairs?status=	按状态筛选（支持中文与 0/1 两种写法）

分页响应（真实抓取）

```
{"code":0,"message":"ok","data":{"total":6,"page":1,"size":3,"pages":2,
  "list":[{"id":7,"account":"stu006","name":"孙八","sex":"女",
           "phone":"13800000006","room":"308"}]}}
```

新增报修响应（含后端联表结果）

```
{"code":0,"message":"ok","data":{"id":4,"userId":2,"thing":"台灯",
  "cause":"不亮了","status":0,"statusText":"未修",
  "orderDate":"2026-11-02 19:25:22","date":"",
  "studentName":"张三","studentRoom":"103"}}
```

studentName / studentRoom 不属于 repairs 表，是后端 join user 算出来的 —— 客户端一次请求即可拿到，无需二次查询。

异常响应

```
{"code":400,"message":"thing 不能为空"}
{"code":400,"message":"status 只能是 未修/已修（或 0/1）"}
{"code":404,"message":"接口不存在：nope"}
{"code":405,"message":"不支持该请求方法：POST"}
{"code":503,"message":"数据库连不上：请确认 MySQL 已启动（端口 3306）"}
```

自动化测试

客户端 3 个 JVM 单元测试（无需模拟器）：用**真实后端返回的 JSON** 验证 DTO 字段映射。 Gson 按字段名反射填充，字段名写错一个字母就得到 null 且不报错 —— 这条测试守的是“客户端 DTO 与后端格式”的契约。

```
NetMappingTest
  OK studentsResponse_mapsEveryField
  OK repairResponse_includesJoinedUserFields
  OK errorResponse_mapsCodeAndMessage

BUILD SUCCESSFUL - 3 tests, 0 failures
```

数据源与安全策略

原则：服务端是唯一权威数据源。客户端可以有本地数据，但它只是"暂存"，不参与裁决。

角色	职责
服务端 • MySQL	**唯一权威。**写入、状态流转、时间戳全部由它决定
客户端 • Room	仅作**本地暂存 / 离线展示**；联网后以服务端返回为准，**不做本地优先**
冲突处理	以服务端最新数据覆盖本地，客户端不参与判断谁对

为什么"服务端权威"顺带解决了安全问题

风险	做法	证据
SQL 注入	全部 SQL 走 `#{}` 预编译占位符，**全项目 0 处 `${}` 字符串拼接**	mapper/*.xml
客户端伪造字段	`status` / `date` / `orderDate` 由服务端赋值，客户端传了也无效	RepairsService.create
越权提交	服务端校验 `userType == 学生` 才允许提交报修（管理员 id 会被拒）	RepairsService.create
本地库被篡改	本地只是缓存，改它不影响权威数据；下次同步即被服务端覆盖	设计原则

> 一句话：**客户端不可信，所以它只负责"显示"，不负责"决定"。**
> 凡是"谁的数据算数""这个状态能不能变""时间取哪个"这类问题，答案一律在服务端。

落地：读和写用的是不同策略

操作	策略	为什么这么选
读 · 学生列表	网络优先 → 成功则**整体覆盖**本地缓存 → 失败才回退缓存	离线也能看一眼，但界面会明确标注「数据来源：本地缓存，可能过期」
写 · 提交报修	**只走服务端，不做缓存回退**	提交失败必须让用户知道失败——不能用本地"假装成功"

这个不对称是刻意的：**读可以降级，写不能降级。**让写操作"看起来成功"是最危险的一类 bug。

数据库升级：第一个真正的 Migration

新增缓存表需要把库从 v8 升到 v9。之前 v7→v8 只能用 `fallbackToDestructiveMigration()`（旧库是 SQLiteOpenHelper 手写的，没有 Room 的身份校验表，没法忠实迁移）； v8 之后库已是 Room 建的，所以这次写了**精确迁移**：只加一张表，不动老数据。

```
static final Migration MIGRATION_8_9 = new Migration(8, 9) {
    public void migrate(SupportSQLiteDatabase db) {
        db.execSQL("CREATE TABLE IF NOT EXISTS `cached_student` (...)");   // 不动其它表
```

```
    }
};
```

当前进度（诚实说明）

新增的联调页已按上述策略完整实现（网络优先 + 缓存回退 + 界面标注来源）。 存量 20+ 页面仍是纯本地读取，尚未收敛到 Repository——这是**分步推进**，不是遗漏。

界面截图



登录页 · 学生 / 管理员统一入口



管理员首页 · 7 个管理模块



请假审批 · 待审核 / 已同意 / 已驳回



驳回 · 必须填写驳回原因

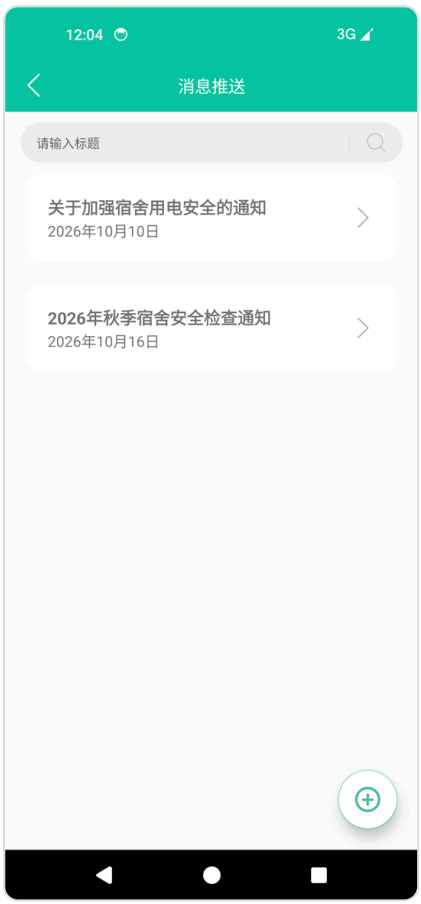


报修处理 · 一键标记已修

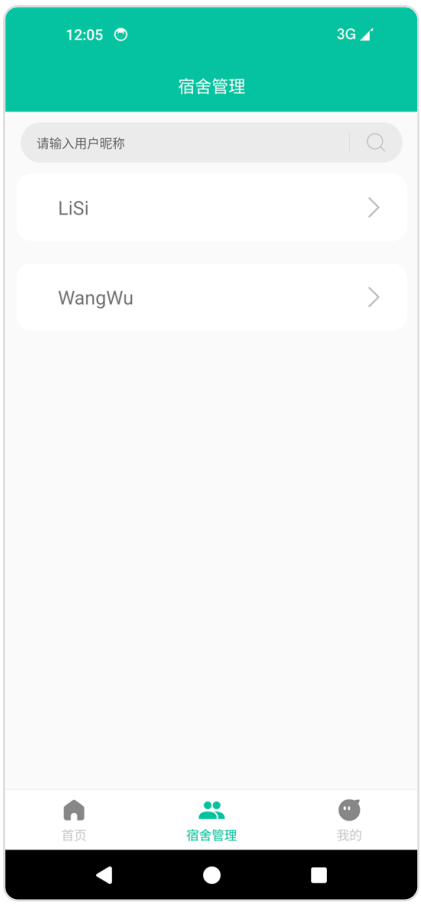


卫生评分 · 按寝室录入

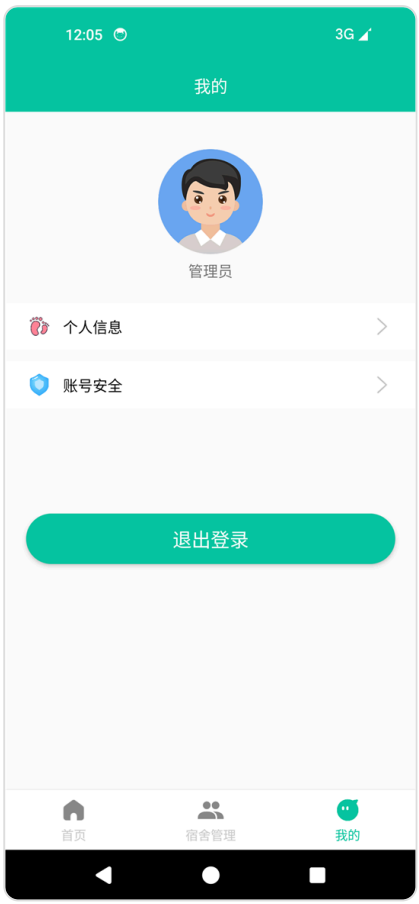
界面截图（2/4）



公告 · 发布与查看



宿舍管理 · 学生增删改查



管理员 · 我的



管理员独立登录入口



学生首页 · 7 个功能入口



请假上报 · 表单 + 本人记录与审批状态

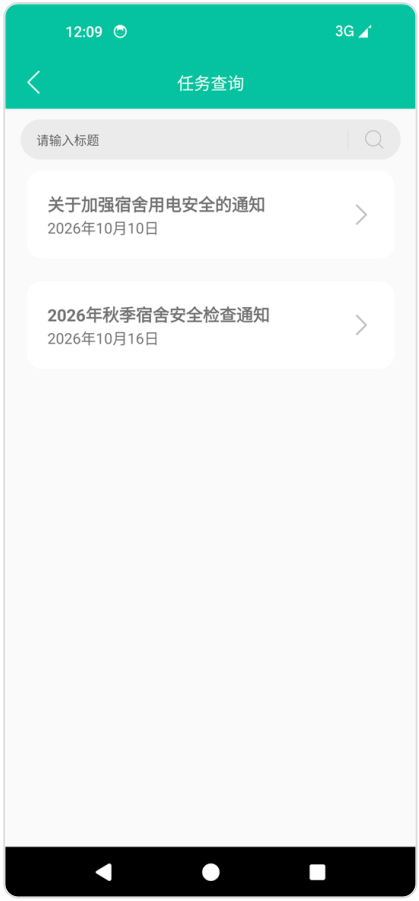
界面截图（3/4）



欠费查询 · 年费从 room_type 表读取



扫码支付 · 收款码



公告列表



评分列表



出入打卡



学生 · 我的

界面截图（4/4）



缴费管理 · 登记欠费 + 查看记录

已知边界

下面这些是主动说明的，不是隐瞒：

存量页面未收敛	存量 20+ 页面仍直读 Room；统一到 Repository 层（服务端权威 + 本地缓存）是下一步，新增联调页已按该策略实现
密码明文	未做加盐哈希，教学项目取舍
测试覆盖有限	有 3 个网络层 JSON 映射测试；UI 与数据库层仍无自动化测试
主线程查库	Room 默认禁止，本项目用 allowMainThreadQueries() 放开，属有意的技术债
后端无鉴权	接口裸奔，没有登录 / token
破坏性迁移	升级用 fallbackToDestructiveMigration()，旧库升级会清空数据

值得讲的两个真实缺陷修复

① 一个查询静默返回空，完全不报错。宿舍管理列表永远是空的，但同一条 SQL 在设备上手动执行却能查出数据。 根因是建表时列没有声明类型 —— SQLite 的亲和性由声明类型决定，没声明就是 NONE 亲和性、不做类型转换， 于是 `where userType = ?` 绑定文本 "0" 时，TEXT 和 INTEGER 匹配不上。 更反直觉的是：这个坑恰恰是“有人为了安全改用占位符”那一刻才爆发的。

② 数据库连不上只报“500 服务器内部错误”。调用方完全看不出是 MySQL 没启动。 加了 DataAccessException → 503 的独立处理器，把原因直接写进响应体； 同时确认了一个容易误解的行为 —— MySQL 中途重启后，后端不用重启也能自动恢复（连接池会重建连接）。